

Labo 00 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — Un binaire Linux tourne partout... sauf sur Windows

Réponse. Un binaire ne demande rien à « Ubuntu » ou à « Alpine » : il demande tout au **noyau**, via les appels système (`open`, `read`, `fork`...). Ces appels sont identiques sur toute machine Linux : la distribution ne fournit que l'espace utilisateur autour. Le noyau Windows expose d'autres appels, incompatibles : le binaire n'y a pas d'interlocuteur.

Pourquoi. L'interface noyau ↔ programmes est stable et standardisée (Linux s'interdit de la casser). Tout ce qui distingue les distributions — gestionnaire de paquets, versions de bibliothèques, configuration — vit *au-dessus* de cette interface.

Nuance. « Identique » suppose que les bibliothèques dynamiques nécessaires soient présentes (la `libc` par exemple, différente entre Debian et Alpine — vous y serez confronté au labo 05). Et WSL ne « traduit » pas : WSL 2 fait tourner un **vrai** noyau Linux dans une VM.

Exemple.

```
uname -r          # 6.6.87.2-microsoft-standard-WSL2 : un vrai noyau Linux, signé Microsoft
uname -m          # x86_64 : l'architecture, l'autre condition de compatibilité
```

Question 2 — L'orphelin adopté par PID 1

Réponse. Le shell (parent du `sleep`) est mort avec le terminal. Le noyau ne laisse jamais un processus sans parent : l'orphelin est **adopté** par PID 1 (`systemd`), d'où le PPID = 1. `sleep` continue de tourner normalement.

Pourquoi. Le parent a un rôle précis : quand un enfant meurt, c'est lui qui lit son code de sortie (l'enfant reste sinon en état « zombie »). Il faut donc toujours un tuteur de dernier recours — c'est l'une des responsabilités de PID 1.

Nuance. C'est exactement pour cela que le PID 1 *d'un conteneur* est un sujet sérieux (labo 03) : votre application y hérite de ce rôle de tuteur sans le savoir, et un PID 1 qui n'enterre pas ses zombies ou n'a aucun parent de secours change le comportement du conteneur.

Exemple.

```
sleep 300 &
ps -o pid,ppid,cmd | grep [s]leep    # 2419 2363 sleep 300 (PPID = votre bash)
# fermez le terminal, rouvrez-en un :
ps -ef | grep [s]leep                # ubuntu 2419 1 ... sleep 300 (PPID = 1)
```

Question 3 — Permission denied, code 126

Réponse. Le fichier n'a pas le bit d'exécution `x`. Confirmation : `ls -l deploy.sh` (on lit `-rw-r--r--`, pas de `x`). Correction : `chmod +x deploy.sh`. Contournement sans rien corriger : `bash deploy.sh` — c'est alors `bash` (exécutable, lui) qui est lancé, le script n'étant qu'un argument lu.

Pourquoi. Lancer `./deploy.sh` demande au noyau d'**exécuter ce fichier** ; le noyau vérifie le bit `x` et refuse. Le code 126 est la convention du shell : « trouvé, mais non exécutable » — à distinguer de 127, « pas trouvé ».

Nuance. Un fichier créé par un éditeur ou téléchargé naît en `rw-` : l'exécution est un droit qui s'ajoute volontairement. Dans un Dockerfile (labo 04), le `COPY` d'un script suivi d'un `RUN ./script.sh` échouera de la même façon si le bit `x` manquait dans votre dépôt Git.

Exemple.

```
printf '#!/bin/bash\nnecho bonjour\n' > deploy.sh
./deploy.sh ; echo $?      # bash: ./deploy.sh: Permission denied ; 126
chmod +x deploy.sh
./deploy.sh                # bonjour
```

Question 4 — Variable de shell vs variable d'environnement

Réponse. Ligne 1 : `1:` (vide). Ligne 2 : `2: bonjour`. Avant `export`, `MSG` n'est qu'une variable **du shell courant** ; le `bash -c` enfant naît avec l'environnement hérité, où `MSG` n'existe pas. Après `export`, `MSG` entre dans l'environnement, et tout enfant en reçoit une copie.

Pourquoi. À sa création, un processus reçoit une **copie** de l'environnement de son parent, jamais une référence : c'est un héritage à sens unique, figé à l'instant du lancement.

Nuance. Les guillemets **simples** de `'echo 1: $MSG'` sont essentiels : ils empêchent votre shell de remplacer `$MSG` avant de lancer l'enfant. Avec des guillemets doubles, les deux lignes afficheraient `bonjour` ... mais ce serait le parent qui aurait fait la substitution, pas l'enfant. Corollaire important : modifier l'environnement d'un processus **déjà lancé** est impossible — d'où le `podman run -e` fixé au démarrage (labo 08).

Exemple.

```
MSG=bonjour
env | grep MSG      # rien
export MSG
env | grep MSG      # MSG=bonjour
```

Question 5 — Le code 137

Réponse. $137 = 128 + 9$: le processus est mort en recevant le signal numéro 9, `SIGKILL`. Personne ne lui a laissé la moindre chance de s'arrêter proprement. Ce code est célèbre car c'est celui d'un conteneur abattu : par `docker kill`, par un `docker stop` resté sans réponse après son délai de grâce, ou par l'**OOM killer** du noyau quand la mémoire manque.

Pourquoi. Le shell encode la mort par signal en `128 + numéro` pour la distinguer d'un `exit n` volontaire. `SIGKILL` n'est jamais délivré au processus : le noyau le supprime directement, sans exécution de code de nettoyage.

Nuance. Diagnostiquer un 137 demande donc de chercher **qui** a envoyé le 9 : un humain, un orchestrateur... ou le noyau. `dmesg | grep -i "out of memory"` tranche le cas OOM. Vous ferez ce diagnostic sur de vrais conteneurs au labo 03.

Exemple.

```
bash -c 'kill -9 $$' ; echo $? # 137 ($$ = le PID du bash enfant lui-même)
sleep 300 & kill -9 $! ; wait $! ; echo $? # 137 aussi
```

Question 6 — kill poli, kill -9 brutal

Réponse. kill (donc SIGTERM) est une **demande** : le processus la reçoit, peut exécuter son code d'arrêt — vider ses tampons, clore ses transactions, fermer ses connexions — puis sortir. kill -9 (SIGKILL) n'est pas délivré au processus : le noyau l'efface immédiatement. Une base de données perd alors tout ce qui n'était pas écrit sur disque et devra rejouer son journal au redémarrage — voire réparer des fichiers corrompus.

Pourquoi. C'est précisément le protocole de docker stop : envoi de SIGTERM au PID 1 du conteneur, délai de grâce (10 s par défaut), puis SIGKILL si le processus n'a pas obtempéré. Une application qui ignore SIGTERM est donc **toujours** tuée brutalement au bout du délai.

Nuance. kill -9 a sa place : processus bloqué qui ignore réellement SIGTERM. Le réflexe correct est l'escalade — kill, attendre, puis kill -9 — jamais l'inverse. Notez aussi que SIGKILL ne peut être ni capté ni ignoré : c'est le seul recours garanti.

Exemple.

```
sleep 300 &
kill %1 # SIGTERM : le job affiche "Terminated"
sleep 300 &
kill -9 %1 # SIGKILL : le job affiche "Killed"
```

Question 7 — Lire -rw-r----- root shadow

Réponse. Les neuf bits se découpent en trois triplets : propriétaire rw-, groupe r--, autres --. Votre utilisateur n'est ni root (propriétaire) ni membre du groupe shadow : il tombe dans « autres », qui n'a **aucun** droit — d'où le refus. sudo cat exécute cat avec l'UID 0, et le noyau n'applique pas les contrôles de permission à root. Sans sudo, seuls root et les membres du groupe shadow (en lecture) peuvent lire le fichier.

Pourquoi. À chaque open, le noyau compare l'UID/GID du processus appelant aux bits du fichier : propriétaire d'abord, sinon groupe, sinon « autres ». Le premier triplet applicable est le seul appliqué.

Nuance. /etc/shadow contient les empreintes des mots de passe — c'est le fichier d'exemple canonique. Notez que la règle « premier triplet applicable » peut surprendre : un fichier ----rw-rw- serait illisible... par son propre propriétaire.

Exemple.

```
id # uid=1000(ubuntu) ... : ni root, ni groupe shadow
cat /etc/shadow # Permission denied, code 1
sudo head -n 1 /etc/shadow # root:!:20501:0:99999:7:::
```

Question 8 — Deux flux, deux fichiers

Réponse. L'écran n'affiche **rien**. resultat.txt contient la ligne du succès (/etc/hostname) ; erreurs.txt contient le message ls: cannot access '/date-inconnue': No such file or

directory. Avec `> resultat.txt 2>&1`, les deux lignes iraient dans `resultat.txt` et `erreurs.txt` ne serait pas créé.

Pourquoi. `ls` écrit ses résultats sur **stdout** (flux 1) et ses plaintes sur **stderr** (flux 2). `>` ne détourne que le flux 1, `2>` que le flux 2 ; `2>&1` signifie « fais pointer le flux 2 là où pointe le flux 1 maintenant ».

Nuance. L'ordre compte : `2>&1 > resultat.txt` enverrait les erreurs... à l'écran (le flux 2 est branché sur l'ancien flux 1 avant la redirection). Cette séparation des flux est ce qui permettra à `podman logs` de vous montrer erreurs et sortie normale d'un conteneur (labo 03).

Exemple.

```
ls /etc/hostname /date-inconnue > resultat.txt 2> erreurs.txt
cat resultat.txt      # /etc/hostname
cat erreurs.txt       # ls: cannot access '/date-inconnue': No such file or directory
```

Question 9 — 127.0.0.1 vs 0.0.0.0

Réponse. Redis écoute sur `127.0.0.1:6379` : uniquement l'interface *loopback*, donc joignable **seulement depuis la machine elle-même**. Java écoute sur `0.0.0.0:8080` : toutes les interfaces, donc joignable depuis le réseau. D'un autre poste, seul le service Java répond.

Pourquoi. L'adresse d'écoute est un filtre : le noyau ne remet au processus que les connexions arrivées sur cette adresse. `0.0.0.0` signifie « toutes les adresses de la machine ».

Nuance. C'est une frontière de sécurité de premier ordre : une base de données en écoute locale n'est pas attaquable du réseau. Au labo 07, vous verrez que `podman run -p 8080:80` publie sur `0.0.0.0` par défaut — et que `-p 127.0.0.1:8080:80` restreint volontairement. Comprendre ces deux lignes de `ss`, c'est déjà comprendre `-p`.

Exemple.

```
python3 -m http.server 8080 --bind 127.0.0.1 &
ss -tlnp | grep 8080      # LISTEN ... 127.0.0.1:8080 ... ("python3",pid=...)
kill %1
```

Question 10 — Le port 80 refusé

Réponse. Les ports **inférieurs à 1024** (dits *priviliés*) ne peuvent être ouverts que par root (UID 0). Votre processus, UID 1000, se voit refuser le `bind` sur le port 80 ; 8080 est au-dessus du seuil, donc libre. Historiquement, la règle garantissait que sur une machine partagée, un service « officiel » (port 25, 80...) ne pouvait pas être usurpé par un simple utilisateur. Conséquence : Podman rootless, simple processus à votre UID, ne peut pas publier `-p 80:80` — on publie `-p 8080:80` à la place.

Pourquoi. Le contrôle est fait par le noyau au moment de l'appel système `bind`, sur la base de l'UID effectif (plus précisément d'une *capability* que root possède).

Nuance. Le seuil est réglable (`sysctl net.ipv4.ip_unprivileged_port_start`), et les vrais serveurs web de production tournent derrière un répartiteur qui, lui, possède le port 80. Le message d'erreur de Podman (`pasta failed ... Permission denied`) reviendra au labo 07.

Exemple.

```
python3 -m http.server 80
# PermissionError: [Errno 13] Permission denied
python3 -m http.server 8080 & # fonctionne
kill %1
```

Question 11 — `/proc`, le faux répertoire

Réponse. `/proc` est un **système de fichiers virtuel** (type `proc`), monté sur `/proc`, dont le contenu n'existe sur aucun disque : chaque lecture est fabriquée à la volée par le noyau à partir de son état interne. Les répertoires numériques sont les processus vivants ; les autres fichiers décrivent le système. Exemples d'usage : `/proc/<pid>/environ` (l'environnement réel d'un processus), `/proc/meminfo` (la mémoire), `/proc/self/uid_map` (les correspondances d'UID — la preuve du rootless au labo 01).

Pourquoi. « Tout est fichier » : exposer l'état du noyau sous forme de fichiers permet d'utiliser `cat`, `grep` et `ls` comme outils d'administration, sans API dédiée. `ps` n'est qu'un habillage de `/proc`.

Nuance. C'est aussi pourquoi un conteneur reçoit **son propre** `/proc` monté à sa création : sinon il verrait tous les processus de l'hôte. Quand `ps` ment dans un conteneur, c'est que son `/proc` est isolé — pas que les processus ont disparu.

Exemple.

```
findmnt -t proc          # /proc proc proc rw,relatime
df -h /proc 2>/dev/null # aucun disque associé
tr '\0' '\n' < /proc/self/environ | head -3 # l'environnement... de ce cat
```

Question 12 — `command not found`, code 127

Réponse. Le shell a parcouru, dans l'ordre, chaque répertoire de la variable `PATH` à la recherche d'un exécutable `monoutil` ; `~/outils` n'y figure pas, la recherche échoue, code 127. Immédiat : lancer par chemin explicite, `~/outils/monoutil`. Durable : (1) ajouter le répertoire au `PATH` dans `~/.bashrc` (`export PATH="$HOME/outils:$PATH"`), ou (2) copier/liier l'outil dans un répertoire déjà présent, comme `~/local/bin` ou `/usr/local/bin`.

Pourquoi. Le `PATH` est le seul mécanisme de résolution des commandes « nues ». Le répertoire courant n'en fait volontairement pas partie : un `ls` piégé déposé dans `/tmp` ne doit pas s'exécuter parce que vous avez fait `cd /tmp`.

Nuance. 127 (introuvable) et 126 (trouvé mais non exécutable) sont deux diagnostics distincts. Dans un conteneur, l'erreur `exec: "monoutil": executable file not found in $PATH` a exactement la même cause — le `PATH` de l'image (labo 04).

Exemple.

```
mkdir -p ~/outils && printf '#!/bin/bash\nnecho ok\n' > ~/outils/monoutil && chmod +x
~/outils/monoutil
monoutil          # command not found ; echo $? → 127
export PATH="$HOME/outils:$PATH"
monoutil          # ok
```

Question 13 — Pourquoi les 12-factor aiment l'environnement

Réponse. Parce que l'environnement est attaché au **processus**, pas à la machine : il est fixé au lancement, hérité automatiquement, et disparaît avec le processus. Pour des applications jetables et relançables, cela donne une configuration (1) injectable de l'extérieur sans modifier ni le code ni les fichiers livrés, (2) différente par instance — deux processus côte à côte avec deux configurations, (3) sans état résiduel : relancer avec d'autres valeurs suffit, rien à nettoyer.

Pourquoi. L'héritage parent → enfant fait tout le travail : celui qui lance (le shell, systemd, plus tard le moteur de conteneurs) prépare le dictionnaire, l'application ne fait que lire. Le même artefact — JAR ou image — passe d'un environnement à l'autre inchangé ; seule la « dot » de variables change.

Nuance. L'immutabilité de l'héritage est aussi sa limite : changer une variable impose de **relancer** le processus. C'est un non-problème pour un conteneur (jetable par conception), mais c'est un vrai deuil pour qui espérait reconfigurer à chaud. Les secrets, eux, demanderont mieux que des variables visibles dans `/proc/<pid>/environ` (labo 08).

Exemple.

```
SERVER_PORT=9090 java -jar app.jar    # même JAR, autre port – rien n'a été modifié
# ce sera, mot pour mot :
# podman run -e SERVER_PORT=9090 mon-api
```